Name: Alizay Aman Niazi.

Sap ID: 58604.

Section: BSSE-5.

Course: MAD.

Lab Task 9

LAB TASK:

database_helper.dart:

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import 'models.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._internal();
  static Database? _database;
  DatabaseHelper._internal();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, 'todo_manager.db');
    return await openDatabase(path, version: 1, onCreate: _createDatabase);
  }
```

```
Future<void> _createDatabase(Database db, int version) async {
  await db.execute('''
    CREATE TABLE categories (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL
    )''');

  await db.execute('''
    CREATE TABLE tasks (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      description TEXT,
      categoryId INTEGER,
      isDone INTEGER NOT NULL,
      FOREIGN KEY (categoryId) REFERENCES categories (id) ON DELETE CASCADE
    )''');

  // Default categories
  await db.insert('categories', {'name': 'Work'});
  await db.insert('categories', {'name': 'Personal'});
}

// Category CRUD
Future<int> addCategory(CategoryModel category) async =>
    (await database).insert('categories', category.toJson());

Future<List<CategoryModel>> getCategories() async {
  final res = await (await database).query('categories');
  return res.map((json) => CategoryModel.fromJson(json)).toList();
}

Future<int> deleteCategory(int id) async =>
    (await database).delete('categories', where: 'id = ?', whereArgs: [id]);

// Task CRUD
Future<int> addTask(TaskModel task) async =>
    (await database).insert('tasks', task.toJson());

Future<List<TaskModel>> getTasks() async {
  final db = await database;
```

```
    final res = await db.rawQuery('''
      SELECT tasks.*, categories.name as categoryName
      FROM tasks
      JOIN categories ON tasks.categoryId = categories.id
    ''');
    return res.map((json) => TaskModel.fromJson(json)).toList();
  }

  Future<int> updateTask(TaskModel task) async => (await database)
      .update('tasks', task.toJson(), where: 'id = ?', whereArgs: [task.id]);

  Future<int> deleteTask(int id) async =>
      (await database).delete('tasks', where: 'id = ?', whereArgs: [id]);
}
```

models.dart:

```
class CategoryModel {
  final int? id;
  final String name;

  CategoryModel({this.id, required this.name});

  Map<String, dynamic> toJson() => {'id': id, 'name': name};

  factory CategoryModel.fromJson(Map<String, dynamic> json) =>
      CategoryModel(id: json['id'], name: json['name']);
}

class TaskModel {
  int? id;
  final String title;
  final String description;
  final int categoryId;
  final bool isDone;
  final String? categoryName; // Helper for UI display

  TaskModel({
    this.id,
    required this.title,
    required this.description,
```

```
    required this.categoryId,
    this.isDone = false,
    this.categoryName,
  });

  Map<String, dynamic> toJson() => {
    'id': id,
    'title': title,
    'description': description,
    'categoryId': categoryId,
    'isDone': isDone ? 1 : 0,
  };

  factory TaskModel.fromJson(Map<String, dynamic> json) => TaskModel(
    id: json['id'],
    title: json['title'],
    description: json['description'],
    categoryId: json['categoryId'],
    isDone: json['isDone'] == 1,
    categoryName: json['categoryName'],
  );
}
```

main.dart:

```
import 'package:flutter/material.dart';
import 'database_helper.dart';
import 'models.dart';

void main() => runApp(const TodoApp());

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple, primary:
Colors.deepPurple),
        useMaterial3: true,
```

```
      ),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _currentIndex = 0;
  List<TaskModel> _tasks = [];
  List<CategoryModel> _categories = [];

  @override
  void initState() {
    super.initState();
    _refreshData();
  }

  void _refreshData() async {
    final t = await DatabaseHelper.instance.getTasks();
    final c = await DatabaseHelper.instance.getCategories();
    setState(() {
      _tasks = t;
      _categories = c;
    });
  }

  // Dialog to Add/Edit Task
  void _showTaskDialog({TaskModel? task}) {
    final title = TextEditingController(text: task?.title);
    final desc = TextEditingController(text: task?.description);

    // Local variable to track selection inside the dialog
    int? selectedCat = task?.categoryId ?? (_categories.isNotEmpty ? _categories[0].id : null);
```

```
showDialog(
  context: context,
  builder: (context) => StatefulBuilder(
    builder: (context, setDialogState) {
    return AlertDialog(
        title: Text(task == null ? 'New Task' : 'Update Task',
            style: const TextStyle(color: Colors.deepPurple)),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextField(controller: title, decoration: const InputDecoration(labelText: 'Task Title')),
            TextField(controller: desc, decoration: const InputDecoration(labelText: 'Description')),
            DropdownButtonFormField<int>(
              value: selectedCat,
              items: _categories.map((c) => DropdownMenuItem(
                  value: c.id,
                  child: Text(c.name)
              )).toList(),
              onChanged: (v) {
                // This updates the UI specifically inside the dialog
                setDialogState(() => selectedCat = v);
              },
              decoration: const InputDecoration(labelText: 'Category'),
            ),
          ],
        ),
        actions: [
          TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cancel')),
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: Colors.deepPurple),
            onPressed: () async {
              if (title.text.isEmpty || selectedCat == null) return;

              final model = TaskModel(
                id: task?.id,
                title: title.text,
                description: desc.text,
                categoryId: selectedCat!,
                isDone: task?.isDone ?? false,
              );
```

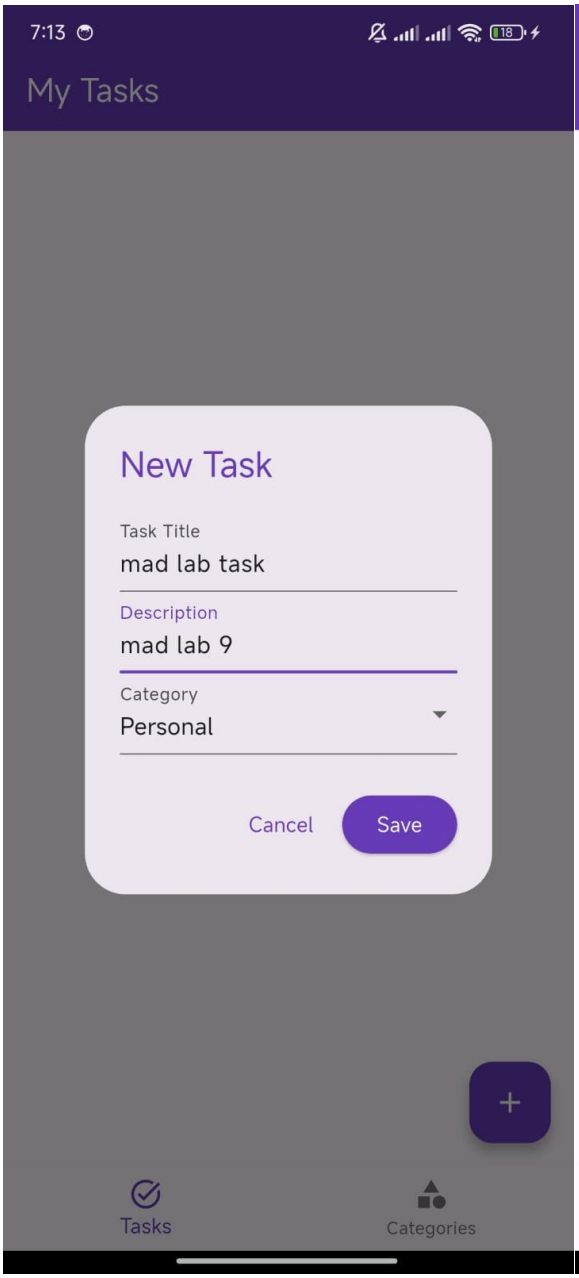
```
            // Add to DB or Update existing
            if (task == null) {
              await DatabaseHelper.instance.addTask(model);
            } else {
              await DatabaseHelper.instance.updateTask(model);
            }

            _refreshData(); // Refresh main list
            Navigator.pop(context); // Close dialog
          },
          child: const Text('Save', style: TextStyle(color: Colors.white)),
        ),
      ],
    );
  },
),
);
}

void _showCategoryDialog() {
  final controller = TextEditingController();
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('New Category', style: TextStyle(color: Colors.deepPurple)),
      content: TextField(controller: controller, decoration: const InputDecoration(hintText: 'e.g.
Study')),
      actions: [
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.deepPurple),
          onPressed: () async {
            if (controller.text.isNotEmpty) {
              // Add category to database
              await DatabaseHelper.instance.addCategory(CategoryModel(name: controller.text));

              _refreshData(); // Refresh categories list so dropdown updates
              Navigator.pop(context);
            }
          },
```

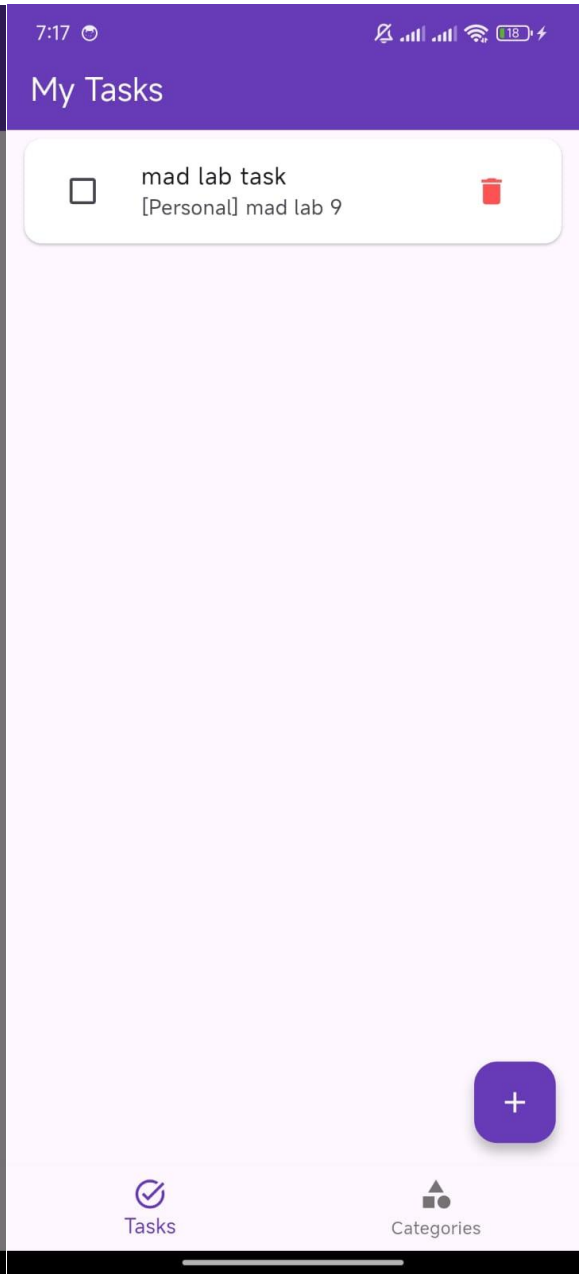
```
        child: const Text('Add', style: TextStyle(color: Colors.white)),
      )
    ],
  ),
);
}
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(_currentIndex == 0 ? 'My Tasks' : 'Categories'),
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
    ),
    body: _currentIndex == 0 ? _buildTaskList() : _buildCategoryList(),
    floatingActionButton: FloatingActionButton(
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
      onPressed: _currentIndex == 0 ? _showTaskDialog : _showCategoryDialog,
      child: const Icon(Icons.add),
    ),
    bottomNavigationBar: BottomNavigationBar(
      currentIndex: _currentIndex,
      selectedItemColor: Colors.deepPurple,
      onTap: (i) => setState(() => _currentIndex = i),
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.task_alt), label: 'Tasks'),
        BottomNavigationBarItem(icon: Icon(Icons.category), label: 'Categories'),
      ],
    ),
  );
}

Widget _buildTaskList() {
  return ListView.builder(
  itemCount: _tasks.length,
  itemBuilder: (context, i) {
      final task = _tasks[i];
      return Card(
        color: task.isDone ? Colors.purple.shade50 : Colors.white,
```

```
        margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        child: ListTile(
          leading: Checkbox(
            value: task.isDone,
            onChanged: (val) async {
              final updated = TaskModel(
                  id: task.id, title: task.title, description: task.description,
                  categoryId: task.categoryId, isDone: val!
              );
              await DatabaseHelper.instance.updateTask(updated);
              _refreshData();
            },
          ),
          title: Text(task.title, style: TextStyle(
              decoration: task.isDone ? TextDecoration.lineThrough : null,
              fontWeight: FontWeight.bold
          )),
          subtitle: Text('[${task.categoryName}] ${task.description}'),
          trailing: IconButton(
            icon: const Icon(Icons.delete, color: Colors.redAccent),
            onPressed: () async {
              await DatabaseHelper.instance.deleteTask(task.id!);
              _refreshData();
            },
          ),
          onTap: () => _showTaskDialog(task: task),
        ),
      );
    },
  );
}

Widget _buildCategoryList() {
  return ListView.builder(
  itemCount: _categories.length,
    itemBuilder: (context, i) => ListTile(
      leading: const Icon(Icons.folder, color: Colors.deepPurple),
      title: Text(_categories[i].name),
      trailing: IconButton(
        icon: const Icon(Icons.delete_outline),
```

```
        onPressed: () async {
          await DatabaseHelper.instance.deleteCategory(_categories[i].id!);
          _refreshData();
        },
      ),
    ),
  );
 }
}
```

OUTPUT:


7:13
My Tasks
New Task
Task Title
mad lab task
Description
mad lab 9
Category
Personal
Cancel
Save
Tasks
Categories


7:17
My Tasks
mad lab task
[Personal] mad lab 9
Tasks
Categories